

Proyecto Unidad II

Sistema de Gestión

Asignatura: Programación Avanzada

Modalidad: Trabajo en Grupos (3 integrantes)

Entrega final y presentación: viernes 30 de octubre, 16:20 hrs.

Presentación de avance obligatorio: viernes 23 de octubre, 16:20 hrs.

Cualquier indicio de copia o uso de IA será evaluado con la nota mínima. No se calificará código sin informe ni informe sin código. Los grupos que no realicen la presentación oral no tendrán evaluado ni su código ni su informe.

1. Contexto y Objetivos

Los sistemas de gestión están en todas partes: una cafetería necesita registrar pedidos y productos; una biblioteca, préstamos y socios; un gimnasio, clases y membresías. Detrás de cada uno existe un motor que organiza información, aplica reglas y mantiene el estado del sistema entre sesiones. Ese motor es, esencialmente, un sistema orientado a objetos bien diseñado.

El objetivo de este proyecto es diseñar e implementar el motor lógico (backend) por consola de un sistema de gestión real. El dominio lo elige el grupo de una lista de temas disponibles. El sistema debe demostrar el uso correcto de los pilares de la programación orientada a objetos estudiados.

1.1. Elección del dominio

A cada grupo se le asigna un tema de la lista provista por el docente. El dominio asignado define las clases concretas del sistema, pero los requisitos técnicos son los mismos para todos los grupos.

Ejemplo: un grupo que elige "Peluquería" implementará un sistema con Servicios (Corte, Tinte, Tratamiento), clientes, estilistas y turnos. Otro grupo que elige "Biblioteca" implementará Materiales (Libro, Revista, DVD), socios y préstamos. Las clases y relaciones cambian, pero los requisitos OOP son idénticos.

1.2. Estructura mínima del sistema

Sin importar el dominio asignado, todo sistema debe contemplar al menos:

- Una clase principal (ej: Producto, Servicio, Material, Reserva) con al menos dos subtipos concretos que hereden de una clase abstracta o implementen una interfaz.
- Una clase de usuario o cliente que interactúe con la clase principal.
- Un registro de operaciones (historial de transacciones, pedidos, préstamos, etc.).
- Un ranking o reporte que ordene información relevante del sistema.

2. Requisitos Técnicos Obligatorios

Estos requisitos aplican a todos los grupos. La elección del dominio no exime de ninguno de ellos.

2.1. Jerarquía de herencia

- Mínimo 2 niveles: una clase abstracta y al menos 2 subclases concretas. Ejemplo: ClaseBase (abstracta) → SubtipoA, SubtipoB, SubtipoC.
- Un tercer nivel es válido y bienvenido si el dominio lo justifica naturalmente (ej: Vehículo → Auto → AutoElectrico), pero no se exige ni se debe forzar.

- Al menos una clase debe ser abstracta con método(s) abstracto(s) que cada subclase implemente de forma distinta (polimorfismo).
- La jerarquía debe ser coherente con el dominio: no se aceptan jerarquías artificiales sin sentido conceptual.

2.2. Interfaces

- Al menos 2 interfaces que definan comportamientos transversales. Ejemplos: Registrable (para clases que se guardan), Rankeable (para clases ordenables), Consultable.
- Las interfaces deben ser implementadas por clases del sistema, no declaradas sin uso real.

2.3. Polimorfismo

- Al menos un método debe invocarse de forma polimórfica sobre una colección de objetos de distintos subtipos. Ejemplo: calcularPrecio(), obtenerDescripcion(), calcularDuracion() — el resultado varía según el subtipo.

2.4. Modificadores static y final

- static: para contadores globales que generen IDs únicos (ej: PROD-001, CLI-001) o para constantes de clase compartidas.
- final: para constantes del sistema (ej: LIMITE_PRESTAMOS = 3, DESCUENTO = 0.10).

2.5. Tipos enumerados (Enum)

- Al menos 2 enums para representar categorías fijas del dominio. Ejemplos: tipo de producto, estado de una reserva, categoría de un servicio.

2.6. Colecciones

- ArrayList: para listas ordenadas de clases (productos, clientes, historial).
- HashMap: para búsqueda eficiente por clave (ej: clientes por RUT, productos por ID).
- Ambas estructuras deben usarse y justificarse en el informe.

2.7. Manejo de excepciones

- Al menos 2 excepciones propias (clases que extiendan Exception o RuntimeException).
- Las excepciones deben lanzarse ante situaciones reales del dominio (ej: stock insuficiente, turno no disponible, monto inválido).
- Deben ser capturadas con try/catch y mostrar mensajes claros al usuario.

2.8. Persistencia

- Toda la información debe guardarse automáticamente al cerrar y cargarse al iniciar.
- El sistema debe quedar exactamente en el mismo estado que tenía antes de cerrar.
- Se acepta persistencia en archivos de texto (CSV/TXT) o serialización de objetos.

3. Funcionalidades Mínimas del Sistema

Independiente del dominio, el sistema debe ofrecer las siguientes funcionalidades operando mediante un menú interactivo con Scanner:

3.1. Gestión de clases principales

- Registrar nueva clase del tipo principal (ej: nuevo producto, nuevo servicio, nuevo material).
- Listar todas las clases con sus atributos relevantes.
- Buscar clase por ID o nombre.
- Ordenar listado por al menos un criterio (nombre, precio, tipo, etc.).

3.2. Gestión de clientes / usuarios

- Registrar nuevo cliente o usuario del sistema.
- Ver perfil del cliente con historial de operaciones.

3.3. Operaciones principales

- Realizar la operación central del dominio (ej: realizar pedido, registrar préstamo, crear reserva, registrar venta).
- Validar restricciones de negocio con excepciones propias (ej: stock, disponibilidad, límites).
- Ver historial de operaciones del sistema.

3.4. Ranking / Reporte

- Mostrar un ranking o reporte ordenado relevante al dominio (ej: productos más vendidos, clientes más activos, servicios más solicitados).

3.5. General

- Salir del sistema guardando el estado completo automáticamente.

4. Datos Precargados Obligatorios

El sistema debe incluir datos precargados que permitan demostrar todas las funcionalidades sin necesidad de ingresarlos manualmente durante la presentación:

- Al menos 3 instancias de cada subtipo de la clase principal (ej: 3 bebidas, 3 comidas, 3 postres).
- Al menos 5 clientes o usuarios registrados.
- Al menos 8 operaciones registradas en el historial (en distintos estados si el dominio los tiene).
- Los datos deben ser coherentes y representativos del dominio elegido.

5. Entregables y Evaluación

Avance obligatorio (viernes 23 de octubre)

El avance debe demostrar: jerarquía de clases implementada (mínimo clase abstracta + 2 subclases), al menos 1 enum funcionando, registro de la clase principal y listado desde menú interactivo, al menos 1 excepción propia. Los datos pueden ser hardcodeados para el avance.

Entrega final (viernes 30 de octubre)

Informe:

- Introducción y descripción del dominio elegido.
- Objetivos y planificación del equipo.
- Diagrama UML completo con todas las clases, interfaces y relaciones.
- Justificación del diseño: por qué la jerarquía elegida es adecuada para el dominio.
- Descripción de las excepciones implementadas y cuándo se lanzan.
- Supuestos y decisiones de diseño del equipo.
- Estructuras de datos utilizadas y justificación (ArrayList vs HashMap).
- Estrategia de persistencia: qué se guarda y cómo.
- Pruebas del sistema y principales resultados.
- Conclusiones.

Código fuente:

- Proyecto desarrollado íntegramente en NetBeans, sin errores de compilación.
- Datos precargados según lo indicado en la Sección 4.

Presentación oral:

- Demostración del sistema con datos precargados.
- Cada integrante debe poder explicar las decisiones de diseño y el código que le corresponde.

6. Ponderación según Syllabus

Componente	Ponderación U2	Tipo
Producto computacional (código)	30%	Grupal
Calidad del código	—	Incluido en producto
Informe del proyecto	10%	Grupal
Presentación oral	20%	Individual